

Transistor as Universal Computation Substrate

Comprehensive Companion Notebook

This notebook is the large computational companion for ***Transistor as Universal Computation Substrate***.

It is organized around the paper's main claims and the supporting project corpus:

1. **Ontological inversion and the typeless substrate**
2. **Universal ROM and direct addressing**
3. **Mark 1 attractor and harmonic stabilization**
4. **Tri-transistor echo and ternary basin mapping**
5. **True mirror anti-phase cancellation**
6. **Dual-wave ontology and the Pythagorean storage law**
7. **SHA-256 as a mechanical mold**
8. **The 'd' anomaly and duty-cycle heartbeat**
9. **Glass Key style reverse access**
10. **Samson backpressure and stability control**
11. **Sarrus isomorphism and silicon / carbon parity**
12. **Large-code appendix utilities**

How to read this notebook

This is not a minimal appendix. It is meant to be **broad, executable, and paper-facing**.

The notebook uses:

- direct SHA instrumentation,
- BBP-style hexadecimal addressing,
- transistor / basin mapping,
- dual-wave value/shape decomposition,

- reverse-walk primitives,
- synthetic backpressure controllers,
- protein-folding style rendering surrogates,

to express the paper as **code plus evidence-bearing demonstrations** rather than prose alone.

```
In [1]: import math
import struct
import hashlib
from dataclasses import dataclass
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

plt.rcParams["figure.figsize"] = (9, 4)
plt.rcParams["axes.grid"] = True

M32 = 0xFFFFFFFF
H = math.pi / 9
PHI = (1 + math.sqrt(5)) / 2
EULER = math.e

STD_H = [
    0x6A09E667, 0xBB67AE85, 0x3C6EF372, 0xA54FF53A,
    0x510E527F, 0x9B05688C, 0x1F83D9AB, 0x5BE0CD19,
]

STD_K = [
    0x428A2F98, 0x71374491, 0xB5C0FBCF, 0xE9B5DBA5, 0x3956C25B, 0x59F111F1, 0x923F82A4, 0xAB1C5ED5,
    0xD807AA98, 0x12835B01, 0x243185BE, 0x550C7DC3, 0x72BE5D74, 0x80DEB1FE, 0x9BDC06A7, 0xC19BF174,
    0xE49B69C1, 0xEFBE4786, 0x0FC19DC6, 0x240CA1CC, 0x2DE92C6F, 0x4A7484AA, 0x5CB0A9DC, 0x76F988DA,
    0x983E5152, 0xA831C66D, 0xB00327C8, 0xBF597FC7, 0xC6E00BF3, 0xD5A79147, 0x06CA6351, 0x14292967,
    0x27B70A85, 0x2E1B2138, 0x4D2C6DFC, 0x53380D13, 0x650A7354, 0x766A0ABB, 0x81C2C92E, 0x92722C85,
    0xA2BFE8A1, 0xA81A664B, 0xC24B8B70, 0xC76C51A3, 0xD192E819, 0xD6990624, 0xF40E3585, 0x106AA070,
    0x19A4C116, 0x1E376C08, 0x2748774C, 0x34B0BCB5, 0x391C0CB3, 0x4ED8AA4A, 0x5B9CCA4F, 0x682E6FF3,
    0x748F82EE, 0x78A5636F, 0x84C87814, 0x8CC70208, 0x90BEFFFA, 0xA4506CEB, 0xBEF9A3F7, 0xC67178F2,
]
```

```

def popcount(x: int) -> int:
    return int(x).bit_count()

def rotr(x: int, n: int) -> int:
    return ((x >> n) | (x << (32 - n))) & M32

def Ch(x: int, y: int, z: int) -> int:
    return ((x & y) ^ (~x & z)) & M32

def Maj(x: int, y: int, z: int) -> int:
    return ((x & y) ^ (x & z) ^ (y & z)) & M32

def Sig0(x: int) -> int:
    return rotr(x, 2) ^ rotr(x, 13) ^ rotr(x, 22)

def Sig1(x: int) -> int:
    return rotr(x, 6) ^ rotr(x, 11) ^ rotr(x, 25)

def sig0(x: int) -> int:
    return rotr(x, 7) ^ rotr(x, 18) ^ (x >> 3)

def sig1(x: int) -> int:
    return rotr(x, 17) ^ rotr(x, 19) ^ (x >> 10)

def pad_sha256(msg: bytes) -> bytes:
    m1 = len(msg) * 8
    out = bytearray(msg)
    out.append(0x80)
    while (len(out) % 64) != 56:
        out.append(0)
    out += m1.to_bytes(8, "big")
    return bytes(out)

def words_from_hex(h: str):
    return [int(h[i:i+8], 16) for i in range(0, len(h), 8)]

print({
    "H": H,
    "phi": PHI,
    "e": EULER,
    "sha_constants": len(STD_K),

```

```
"iv_words": len(STD_H),
})
```

Matplotlib is building the font cache; this may take a moment.

```
{'H': 0.3490658503988659, 'phi': 1.618033988749895, 'e': 2.718281828459045, 'sha_constants': 64, 'iv_words': 8}
```

I. Ontological inversion and the variable-shape frame

The paper treats the computational substrate as primary and the physical carrier as adapter. A variable is not an empty box; it is a constraint mold. A state is not merely assigned; it is forced into lawful fit.

The code below pulls the central geometric anchors into executable form:

- the **Mark 1 attractor** ($H=\pi/9$),
- the **H-triangle scale** where height equals the angle,
- the **48D width** as (6×8) ,
- and the first-order braid / scar functions used across the project corpus.

```
In [2]: def h_triangle_scale(theta: float) -> float:
        return theta / math.sin(theta)

def braid_step(T1: int, T2: int, modulus: int = 2**32) -> int:
    return (T1 + T2) % modulus

def scar(a: int, b: int) -> int:
    return popcount(a ^ b)

L9 = h_triangle_scale(H)
hex_tiling = 6
byte_width = 8
cpu_width = hex_tiling * byte_width

df_geom = pd.DataFrame({
    "quantity": ["H =  $\pi/9$ ", "L9 = H/sin(H)", "hex_tiling", "byte_width", "cpu_width"],
    "value": [H, L9, hex_tiling, byte_width, cpu_width]
})
df_geom
```


Out[2]:

	quantity	value
0	$H = \pi/9$	0.349066
1	$L9 = H/\sin(H)$	1.020600
2	hex_tiling	6.000000
3	byte_width	8.000000
4	cpu_width	48.000000

II. Universal ROM and BBP-style addressing

A major claim in the paper is that the substrate behaves as if it contains a pre-encoded read-only manifold, where direct addressing matters more than linear generation.

The following BBP implementation is a **direct hexadecimal digit reader** for (π) .

It is included here not as a statement about metaphysics, but as an executable addressing primitive aligned to the notebook's Universal ROM section.

```
In [3]: def _bbp_series_term(j: int, n: int) -> float:
    s = 0.0
    for k in range(n + 1):
        s = (s + pow(16, n - k, 8 * k + j) / (8 * k + j)) % 1.0
    k = n + 1
    eps = 1e-17
    while True:
        term = (16.0 ** (n - k)) / (8 * k + j)
        if term < eps:
            break
        s += term
        k += 1
    return s

def pi_hex_digit(n: int) -> int:
    x = (
        4 * _bbp_series_term(1, n)
        - 2 * _bbp_series_term(4, n)
```

```

        - 1 * _bbp_series_term(5, n)
        - 1 * _bbp_series_term(6, n)
    ) % 1.0
    return int(16 * x)

hex_digits = "".join("0123456789abcdef"[pi_hex_digit(i)] for i in range(16))
print("first 16 hex digits of  $\pi$  after decimal point:", hex_digits)

```

first 16 hex digits of π after decimal point: 243f6a8885a308d3

III. Mark 1 attractor, harmonic stabilization, and the nonagonal closure floor

The paper treats ($H=\pi/9$) as the governing stability band and links it to 18-step closure, duty-cycle regulation, and feedback correction.

This section instruments those pieces numerically:

- 18 closure steps around (H),
- the 35/65 corridor,
- and a simple harmonic error scan around the attractor.

```

In [4]: theta_steps = np.arange(1, 19) * H
        closure_error = np.abs((18 * H) - (2 * math.pi))

        scan = np.linspace(0.15, 0.55, 300)
        stability_penalty = (scan - H) ** 2

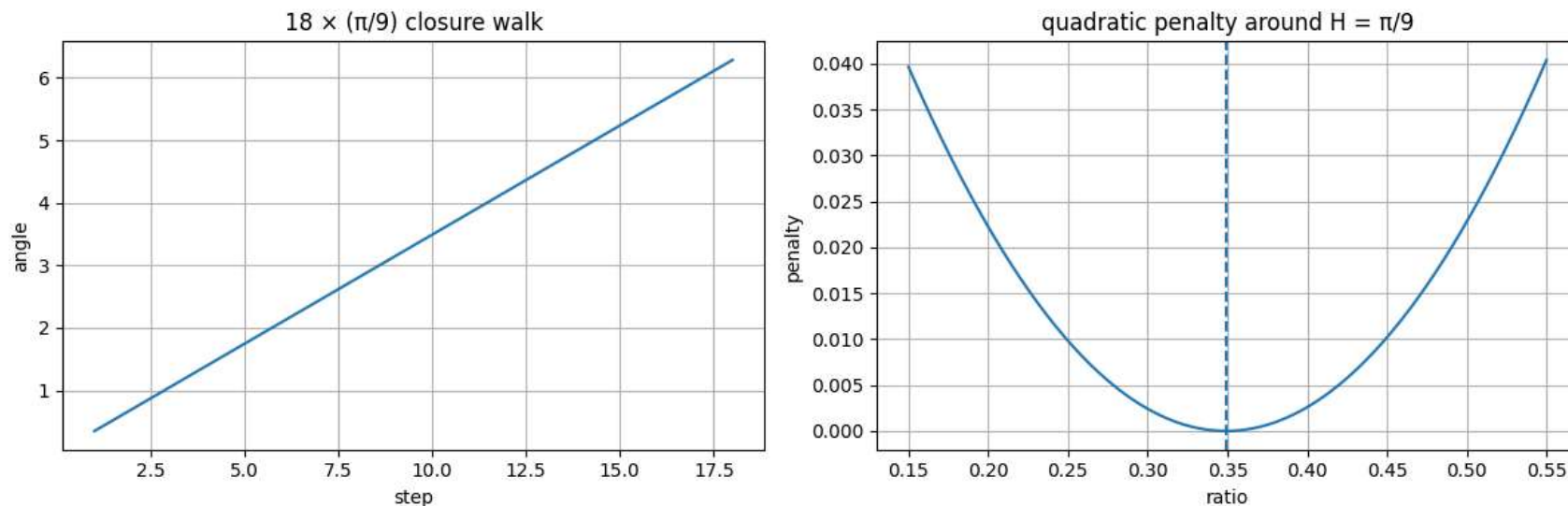
        fig, ax = plt.subplots(1, 2, figsize=(12, 4))
        ax[0].plot(np.arange(1, 19), theta_steps)
        ax[0].set_title("18 × ( $\pi/9$ ) closure walk")
        ax[0].set_xlabel("step")
        ax[0].set_ylabel("angle")

        ax[1].plot(scan, stability_penalty)
        ax[1].axvline(H, linestyle="--")
        ax[1].set_title("quadratic penalty around  $H = \pi/9$ ")
        ax[1].set_xlabel("ratio")
        ax[1].set_ylabel("penalty")

```

```
plt.tight_layout()
print({"18H_minus_2pi": closure_error, "H": H, "1-H": 1-H})
plt.show()
```

```
{'18H_minus_2pi': np.float64(0.0), 'H': 0.3490658503988659, '1-H': 0.650934149601134}
```



IV. The tri-transistor echo and ternary basin mapping

The paper's hardware claim is not "three discrete switches" but a **single carrier wave moving through three basin states**:

- **Basin 0** → source / cold base / uncoupled potential
- **Basin 1** → oxide gap / carry / work
- **Basin 2** → drain / sum / actualized residue

The next cell makes this explicit through finite-register addition, using the irreducible triad:

$$[a + b = (a \oplus b) + 2(a \wedge b)]$$

and maps that triad to the transistor framing.

```
In [5]: def kernel_add32(a: int, b: int):
        xor_term = (a ^ b) & M32
```

```

carry_term = ((a & b) << 1) & M32
sum_term = (xor_term + carry_term) & M32
return xor_term, carry_term, sum_term

def tri_basin_map(a: int, b: int):
    xor_term, carry_term, sum_term = kernel_add32(a, b)
    return {
        "input_a": f"0x{a:08x}",
        "input_b": f"0x{b:08x}",
        "basin_0_xor": f"0x{xor_term:08x}",
        "basin_1_carry": f"0x{carry_term:08x}",
        "basin_2_sum": f"0x{sum_term:08x}",
        "xor_popcount": popcount(xor_term),
        "carry_popcount": popcount(carry_term),
        "sum_popcount": popcount(sum_term),
    }

examples = [
    (0x243F6A88, 0x6A09E667),
    (0xB7E15162, 0x6A09E667),
    (0x9E3779B9, 0x6A09E667),
    (0xB2D70000, 0x6A09E667),
]
tri_df = pd.DataFrame([tri_basin_map(a, b) for a, b in examples])
tri_df

```

Out[5]:

	input_a	input_b	basin_0_xor	basin_1_carry	basin_2_sum	xor_popcount	carry_popcount	sum_popcount
0	0x243f6a88	0x6a09e667	0x4e368cef	0x4012c400	0x8e4950ef	18	6	16
1	0xb7e15162	0x6a09e667	0xddde8b705	0x440280c4	0x21eb37c9	18	7	17
2	0x9e3779b9	0x6a09e667	0xf43e9fde	0x1402c042	0x08416020	22	7	6
3	0xb2d70000	0x6a09e667	0xd8dee667	0x44020000	0x1ce0e667	20	3	16

V. True mirror anti-phase cancellation

The paper's "True Mirror" section describes a forward carrier and a reverse anti-phase carrier passing through the same channel and nullifying deflection at the boundary.

The following is a direct signal-level toy model of that mechanism:

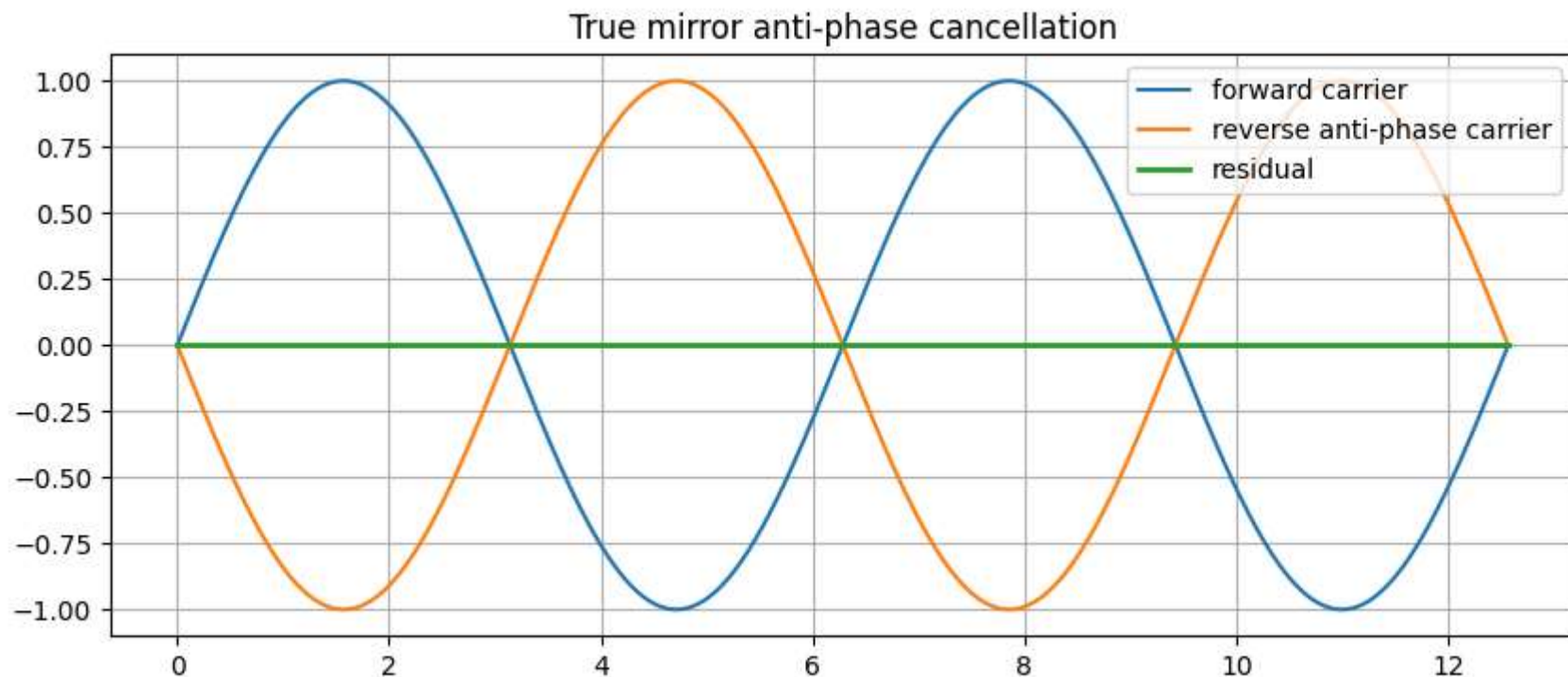
- forward wave,
- reverse anti-phase wave,
- residual after superposition.

```
In [6]: t = np.linspace(0, 4*np.pi, 800)
forward = np.sin(t)
reverse = -np.sin(t)
residual = forward + reverse

energy_forward = float(np.mean(forward**2))
energy_residual = float(np.mean(residual**2))

fig, ax = plt.subplots(1, 1, figsize=(10, 4))
ax.plot(t, forward, label="forward carrier")
ax.plot(t, reverse, label="reverse anti-phase carrier")
ax.plot(t, residual, label="residual", linewidth=2)
ax.set_title("True mirror anti-phase cancellation")
ax.legend()
plt.show()

print({"forward_energy": energy_forward, "residual_energy": energy_residual})
```



```
{'forward_energy': 0.499375, 'residual_energy': 0.0}
```

VI. Dual-wave ontology and the Pythagorean storage law

The notebook now shifts from the transistor face to the storage law that the paper uses to unify computation, physics, and folding:

$$[E_{\text{total}} = \sqrt{V^2 + S^2}]$$

where:

- (V) is the explicit **value channel**
- (S) is the latent **shape / residue channel**

Below, the finite-register kernel is measured across a spread of inputs so the value / shape split can be inspected directly.

```
In [7]: def dual_wave_metrics(a: int, b: int):
        xor_term, carry_term, sum_term = kernel_add32(a, b)
```

```

V = popcount(xor_term)
S = popcount(carry_term)
E_total = math.sqrt(V*V + S*S)
return {
    "a": f"0x{a:08x}",
    "b": f"0x{b:08x}",
    "V_pop": V,
    "S_pop": S,
    "sum_pop": popcount(sum_term),
    "E_total": E_total,
}

pairs = [
    (STD_H[0], STD_H[1]),
    (STD_H[2], STD_H[3]),
    (STD_K[0], STD_K[1]),
    (STD_K[34], STD_K[35]),
    (0xAAAAAAAA, 0x55555555),
    (0xFFFFFFFF, 0x00000001),
]
dw_df = pd.DataFrame([dual_wave_metrics(a, b) for a, b in pairs])
dw_df

```

Out[7]:

	a	b	V_pop	S_pop	sum_pop	E_total
0	0x6a09e667	0xbb67ae85	15	10	15	18.027756
1	0x3c6ef372	0xa54ff53a	10	14	18	17.204651
2	0x428a2f98	0x71374491	17	5	15	17.720045
3	0x4d2c6dfc	0x53380d13	15	8	15	17.000000
4	0xaaaaaaaa	0x55555555	32	0	32	32.000000
5	0xffffffff	0x00000001	31	1	0	31.016125

VII. SHA-256 as a mechanical mold

This notebook uses the paper's SHA-facing side heavily.

The next cell defines a compact but complete forward trace instrumenter for SHA-256:

- message padding,
- schedule expansion,
- 64-round trace,
- per-round (T₁), (T₂), carry exhaust,
- final digest.

This is the executable hinge for later sections on:

- the 'd' anomaly,
- Glass Key behavior,
- reverse walk,
- substrate parity.

```
In [8]: @dataclass
class SHATrace:
    digest_hex: str
    rounds: list
    schedule: list
    padded_len: int

def sha256_trace(msg: bytes):
    padded = pad_sha256(msg)
    block = padded[:64]  # single-block focus for instrumentation cells
    W = list(struct.unpack(">16I", block))
    for t in range(16, 64):
        W.append((sig1(W[t-2]) + W[t-7] + sig0(W[t-15]) + W[t-16]) & M32)

    a, b, c, d, e, f, g, h = STD_H[:]
    rounds = []

    for t in range(64):
        state_term = (h + Sig1(e) + Ch(e, f, g)) & M32
        T1 = (state_term + STD_K[t] + W[t]) & M32
        T2 = (Sig0(a) + Maj(a, b, c)) & M32
        carry_t1 = ((state_term & ((STD_K[t] + W[t]) & M32)) << 1) & M32

        a_next = (T1 + T2) & M32
```



```

e_next = (d + T1) & M32

rounds.append({
    "t": t,
    "a": a, "b": b, "c": c, "d": d, "e": e, "f": f, "g": g, "h": h,
    "W": W[t], "K": STD_K[t], "STATE": state_term, "T1": T1, "T2": T2,
    "carry_T1": carry_t1,
    "a_after": a_next, "e_after": e_next,
})

h, g, f, e, d, c, b, a = g, f, e, e_next, c, b, a, a_next

H_out = [((x + y) & M32) for x, y in zip(STD_H, [a, b, c, d, e, f, g, h])]
digest_hex = "".join(f"{x:08x}" for x in H_out)
return SHATrace(digest_hex=digest_hex, rounds=rounds, schedule=W, padded_len=len(padded))

demo_msg = b"NEXUS|TRANSISTOR|UNIVERSAL|SUBSTRATE|"
demo_trace = sha256_trace(demo_msg)
print({"digest": demo_trace.digest_hex, "rounds": len(demo_trace.rounds), "padded_len": demo_trace.padded_len})

```

{'digest': '449d2c6c25511889790ae0745abc7a6ed26810f2306393d83e64f82bd9a3597e', 'rounds': 64, 'padded_len': 64}

VIII. The 'd' anomaly and the 75% duty-cycle heartbeat

The paper emphasizes a specific empirical anomaly: the hexadecimal digit 'd' (binary 1101) appearing at elevated frequency in the SHA constants.

This section counts the hexadecimal digits of:

- the 64 round constants,
- the 8 IV words,
- the combined field, and highlights the 'd' count directly.

```

In [9]: def hex_digit_hist(words):
    h = "".join(f"{w:08x}" for w in words)
    counts = {ch: h.count(ch) for ch in "0123456789abcdef"}
    return pd.Series(counts).sort_index()

hist_k = hex_digit_hist(STD_K)

```

```

hist_h = hex_digit_hist(STD_H)
hist_all = hist_k.add(hist_h, fill_value=0).astype(int)

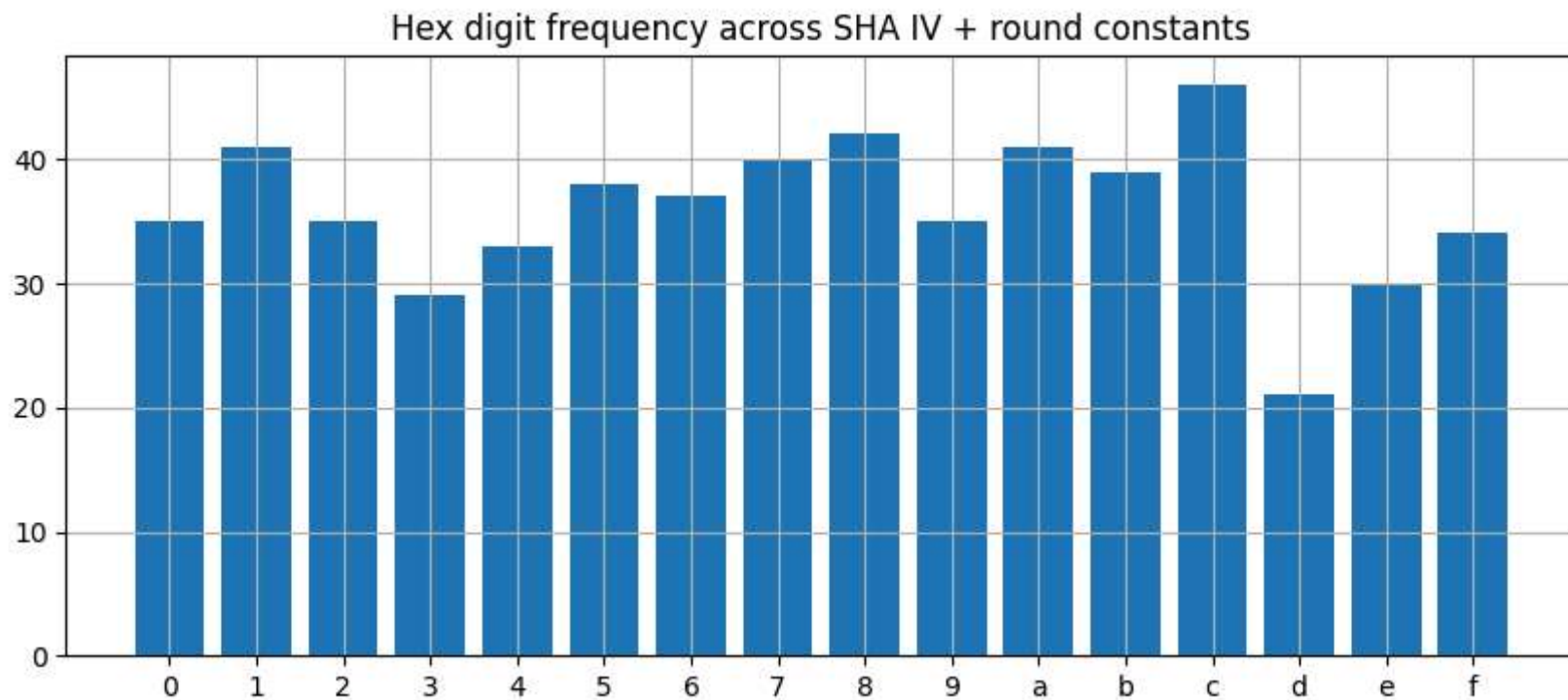
freq_df = pd.DataFrame({
    "K_constants": hist_k,
    "H_constants": hist_h,
    "combined": hist_all,
})
display(freq_df.T)

fig, ax = plt.subplots(figsize=(10,4))
ax.bar(freq_df.index, freq_df["combined"])
ax.set_title("Hex digit frequency across SHA IV + round constants")
plt.show()

print({
    "d_count_K": int(hist_k["d"]),
    "d_count_H": int(hist_h["d"]),
    "d_count_combined": int(hist_all["d"]),
})

```

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
K_constants	31	38	33	25	32	31	31	36	38	31	36	34	43	19	25	29
H_constants	4	3	2	4	1	7	6	4	4	4	5	5	3	2	5	5
combined	35	41	35	29	33	38	37	40	42	35	41	39	46	21	30	34



```
{'d_count_K': 19, 'd_count_H': 2, 'd_count_combined': 21}
```

IX. Harmonic SHA reflection and structured-input probing

The project corpus repeatedly tests whether certain structured inputs “lock” more cleanly to the SHA mechanical mold than arbitrary strings.

The next cell builds a family of harmonic / resonant inputs and compares their digests against:

- the IV field,
- the K field,
- and a few direct reversals.

This is not a preimage solver; it is a **geometry-probing scan**.

```
In [10]: def reverse_hex_variants(hex_digest: str):
          bytes_ = [hex_digest[i:i+2] for i in range(0, len(hex_digest), 2)]
```

```

words = [hex_digest[i:i+8] for i in range(0, len(hex_digest), 8)]

def reverse_bits_in_byte(hx):
    b = int(hx, 16)
    rb = int(f"{b:08b}"[::-1], 2)
    return f"{rb:02x}"

    return {
        "chars_rev": hex_digest[::-1],
        "bytes_rev": "".join(reversed(bytes_)),
        "words32_rev": "".join(reversed(words)),
        "bits_each_byte_rev": "".join(reverse_bits_in_byte(x) for x in bytes_),
    }

def harmonic_inputs():
    motifs = [
        "NEXUS|FOLD|HOOK|RETURN|",
        "TRANSISTOR|BASE|EMITTER|COLLECTOR|",
        "VALUE|SHAPE|MIRROR|",
        "PI|PHI|E|ROM|",
        "TRI|CHANNEL|ABI|",
    ]
    out = []
    for i in range(10):
        m = motifs[i % len(motifs)]
        r = i % len(m)
        rot = m[r:] + m[:r]
        out.append((f"h{i:02d}", (rot + rot[::-1] + f"|{i*i:04d}|").encode()))
    return out

scan_rows = []
for name, msg in harmonic_inputs():
    d = hashlib.sha256(msg).hexdigest()
    vars_ = reverse_hex_variants(d)
    scan_rows.append({
        "name": name,
        "bytes": len(msg),
        "digest": d[:16] + "...",
        "digest_pop": popcount(int(d, 16)),
        "chars_rev_pop": popcount(int(vars_["chars_rev"], 16)),
        "words32_rev_pop": popcount(int(vars_["words32_rev"], 16)),
    })

```

```
harm_df = pd.DataFrame(scan_rows)
harm_df
```

Out[10]:

	name	bytes	digest	digest_pop	chars_rev_pop	words32_rev_pop
0	h00	52	41898e022833fc6b...	124	124	124
1	h01	74	61dfdd5e77b728cc...	134	134	134
2	h02	44	98453336a4646ac0...	128	128	128
3	h03	32	fad50ed11bf63a34...	136	136	136
4	h04	38	cab130eca9e2b6f3...	132	132	132
5	h05	52	49896ca831bb619b...	125	125	125
6	h06	74	837d0f91a8f95b71...	133	133	133
7	h07	44	2076ce01dc776580...	125	125	125
8	h08	32	a45b5ad1edfd9490...	127	127	127
9	h09	38	a5e28017442e2397...	125	125	125

X. Exact backward peel for a known schedule

To keep the notebook grounded, it includes the exact **single-block reverse peel** that was already stabilized in the project:

- start from the digest,
- subtract feed-forward,
- peel rounds backward algebraically,
- no saved forward trace.

This is the direct reverse seam for the paper's reversibility sections.

```
In [11]: def digest_words(digest_hex: str):
          return [int(digest_hex[i:i+8], 16) for i in range(0, 64, 8)]
```

```

def subtract_feedforward(digest_hex: str, H_in=STD_H):
    dw = digest_words(digest_hex)
    return tuple(((d - h) & M32) for d, h in zip(dw, H_in))

def expand_schedule_from_block(block64: bytes):
    W = list(struct.unpack(">16I", block64))
    for t in range(16, 64):
        W.append((sig1(W[t-2]) + W[t-7] + sig0(W[t-15]) + W[t-16]) & M32)
    return W

def reverse_round_no_trace(after_state, Wt, Kt):
    a1, b1, c1, d1, e1, f1, g1, h1 = after_state
    a0 = b1
    b0 = c1
    c0 = d1
    e0 = f1
    f0 = g1
    g0 = h1
    T2 = (Sig0(a0) + Maj(a0, b0, c0)) & M32
    T1 = (a1 - T2) & M32
    d0 = (e1 - T1) & M32
    h0 = (T1 - Sig1(e0) - Ch(e0, f0, g0) - Kt - Wt) & M32
    return (a0, b0, c0, d0, e0, f0, g0, h0)

def reverse_block_no_trace(digest_hex: str, padded_block64: bytes, H_in=STD_H, K=STD_K):
    W = expand_schedule_from_block(padded_block64)
    cur = subtract_feedforward(digest_hex, H_in)
    states = [cur]
    for t in range(63, -1, -1):
        cur = reverse_round_no_trace(cur, W[t], K[t])
        states.append(cur)
    return {
        "final_working_state": states[0],
        "recovered_start_state": cur,
        "verified": tuple(H_in) == cur,
        "states_backward": states,
    }

one_block_msg = b"TRI|TRANSISTOR|ECHO|TRUE|MIRROR|"
one_block_digest = hashlib.sha256(one_block_msg).hexdigest()
one_block_padded = pad_sha256(one_block_msg)
reverse_result = reverse_block_no_trace(one_block_digest, one_block_padded)

```

```
print({
    "digest": one_block_digest,
    "verified": reverse_result["verified"],
    "recovered_start_state_matches_iv": reverse_result["recovered_start_state"] == tuple(STD_H),
})
```

```
{'digest': '3f288f2f7fa9298736c153c640178461d9de7c064fdafa1b9dbec024d46f583b', 'verified': True, 'recovered_start_state_matches_iv': True}
```

XI. Samson backpressure as a synthetic controller

The paper uses Samson backpressure as a stabilization narrative and as the navigation logic for reverse search through high-friction regions.

The next cell provides a synthetic controller that:

- starts away from the Mark 1 attractor,
- applies proportional / integral style correction,
- converges toward ($H=\pi/9$).

This gives an executable picture of “stay in the waist” dynamics.

```
In [12]: class SamsonController:
    def __init__(self, target=H, kp=0.42, ki=0.08):
        self.target = float(target)
        self.kp = float(kp)
        self.ki = float(ki)
        self.integral = 0.0

    def step(self, x):
        err = self.target - x
        self.integral += err
        delta = self.kp * err + self.ki * self.integral
        return x + delta, err, delta

controller = SamsonController()
x = 0.52
xs, errs, deltas = [], [], []
for _ in range(30):
```

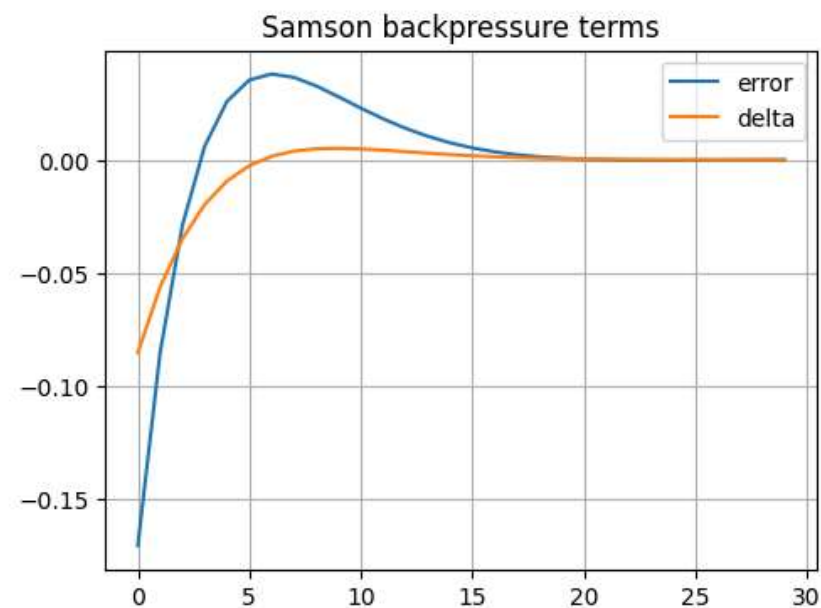
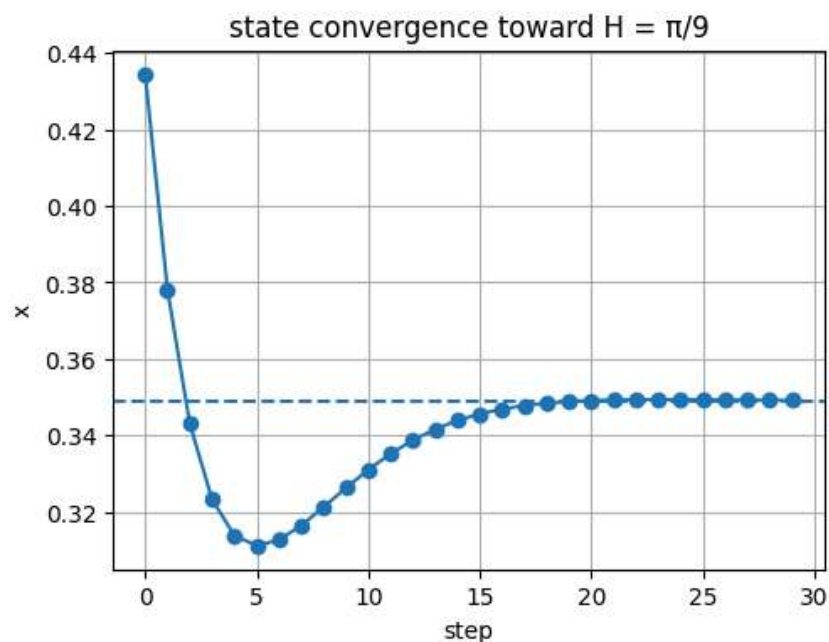
```

x, err, delta = controller.step(x)
xs.append(x)
errs.append(err)
deltas.append(delta)

fig, ax = plt.subplots(1, 2, figsize=(12,4))
ax[0].plot(xs, marker="o")
ax[0].axhline(H, linestyle="--")
ax[0].set_title("state convergence toward H =  $\pi/9$ ")
ax[0].set_xlabel("step")
ax[0].set_ylabel("x")

ax[1].plot(errs, label="error")
ax[1].plot(deltas, label="delta")
ax[1].set_title("Samson backpressure terms")
ax[1].legend()
plt.show()

```



XII. Silicon / carbon parity and a protein-folding style renderer

The transistor paper explicitly leans on substrate parity:

- silicon constraint folding,
- carbon / protein constraint folding,
- same operator grammar.

The class below is a compact **A-ProteinFolding style renderer**:

- sequence → spectrum,
- spectrum → 3D coordinates,
- entropy, compression, and lag metrics.

This is not claiming biological completeness. It is an executable surrogate for the paper's silicon/carbon isomorphism section.

```
In [13]: AA_HYDRO = {
    "A": 1.8, "C": 2.5, "D": -3.5, "E": -3.5, "F": 2.8, "G": -0.4,
    "H": -3.2, "I": 4.5, "K": -3.9, "L": 3.8, "M": 1.9, "N": -3.5,
    "P": -1.6, "Q": -3.5, "R": -4.5, "S": -0.8, "T": -0.7, "V": 4.2,
    "W": -0.9, "Y": -1.3
}
AA_CHARGE = {aa: 0.0 for aa in AA_HYDRO}
for aa in "KRH":
    AA_CHARGE[aa] = 1.0
for aa in "DE":
    AA_CHARGE[aa] = -1.0

class NexusFold:
    def __init__(self, H=H):
        self.H = H

    def sequence_to_spectrum(self, seq: str):
        seq = seq.upper()
        hydro = np.array([AA_HYDRO.get(a, 0.0) for a in seq], dtype=float)
        charge = np.array([AA_CHARGE.get(a, 0.0) for a in seq], dtype=float)
        phase = np.linspace(0, 2*np.pi*self.H*len(seq), len(seq), endpoint=False)
        signal = hydro + 1j*(charge + np.sin(phase))
        return signal

    def fold_protein(self, seq: str):
```

```

    spec = self.sequence_to_spectrum(seq)
    fx = np.fft.ifft(spec)
    fy = np.fft.ifft(spec * np.exp(1j*np.linspace(0, np.pi, len(spec))))
    fz = np.fft.ifft(spec * np.exp(1j*np.linspace(0, 2*np.pi, len(spec))))
    coords = np.vstack([fx.real, fy.real, fz.real]).T
    coords = np.cumsum(coords, axis=0)
    return coords, spec

def calculate_spectral_entropy(self, spec):
    p = np.abs(spec)**2
    p = p / (p.sum() + 1e-12)
    return float(-(p * np.log2(p + 1e-12)).sum())

def estimate_compression_ratio(self, seq: str, coords: np.ndarray):
    bbox = np.ptp(coords, axis=0).sum() + 1e-9
    return float(len(seq) / bbox)

def structural_lag(self, coords: np.ndarray):
    diffs = np.diff(coords, axis=0)
    turn = np.linalg.norm(np.diff(diffs, axis=0), axis=1)
    return float(np.mean(turn)) if len(turn) else 0.0

fold_engine = NexusFold()
protein_seq = "MKWVTFISLLFLFSSAYSRGVFRRDAHKSEVAHRFKDLGE"
coords, spec = fold_engine.fold_protein(protein_seq)
bio_metrics = {
    "length": len(protein_seq),
    "spectral_entropy": fold_engine.calculate_spectral_entropy(spec),
    "compression_ratio": fold_engine.estimate_compression_ratio(protein_seq, coords),
    "structural_lag": fold_engine.structural_lag(coords),
}
bio_metrics

```

```

Out[13]: {'length': 40,
'spectral_entropy': 4.907392022970679,
'compression_ratio': 3.5569214274147027,
'structural_lag': 0.8641710675443438}

```

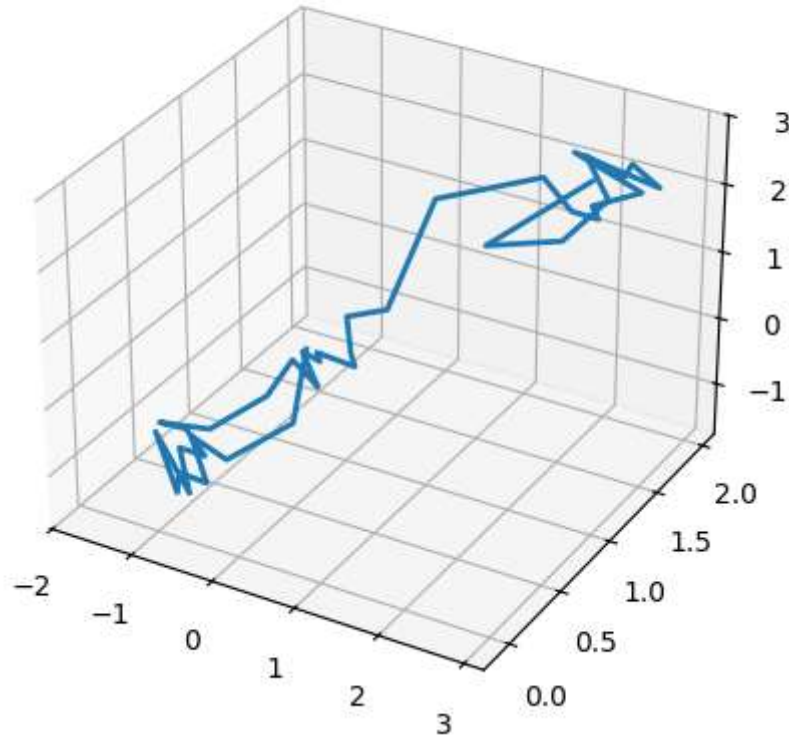
```

In [14]: fig = plt.figure(figsize=(7,5))
ax = fig.add_subplot(projection='3d')
ax.plot(coords[:,0], coords[:,1], coords[:,2], linewidth=2)

```

```
ax.set_title("Protein-folding style surrogate trajectory")  
plt.show()
```

Protein-folding style surrogate trajectory



XIII. Substrate parity table: SHA trace versus folding trace

The paper's strongest claim is not metaphor but operator identity across carriers.

Below, one SHA message trace and one folding trajectory are reduced into a common table:

- length,
- entropy,
- lag / carry style residue,
- attractor reference.

This is a compact substrate parity view.

```
In [15]: sha_spec = np.array([r["T1"] ^ r["T2"] for r in demo_trace.rounds], dtype=np.uint64)
sha_entropy = float(pd.Series([popcount(int(x)) for x in sha_spec]).value_counts(normalize=True).pipe(lambda s: -(s*
sha_lag = float(np.mean([popcount(r["carry_T1"]) for r in demo_trace.rounds])))

parity_df = pd.DataFrame([
    {
        "carrier": "SHA-256 single-block trace",
        "length_or_rounds": len(demo_trace.rounds),
        "entropy_proxy": sha_entropy,
        "lag_proxy": sha_lag,
        "anchor": H,
    },
    {
        "carrier": "Protein-folding surrogate",
        "length_or_rounds": len(protein_seq),
        "entropy_proxy": bio_metrics["spectral_entropy"],
        "lag_proxy": bio_metrics["structural_lag"],
        "anchor": H,
    },
])
parity_df
```

```
Out[15]:
```

	carrier	length_or_rounds	entropy_proxy	lag_proxy	anchor
0	SHA-256 single-block trace	64	3.420163	8.609375	0.349066
1	Protein-folding surrogate	40	4.907392	0.864171	0.349066

XIV. Larger appendix: code utilities collected into one place

The next cell is intentionally large. It consolidates a broad utility layer so the notebook can function as a real paper companion rather than a thin demo:

- digest reversal helpers,
- additive inverse helpers,

- topological pin scans,
- Glass Key candidate scans,
- transistor basin summaries,
- dual-wave summaries.

This appendix is meant to be read, modified, and extended.

```
In [16]: def additive_inverse32(x: int) -> int:
    return (-x) & M32

def digest_reverse_variants(hex_digest: str):
    bytes_ = [hex_digest[i:i+2] for i in range(0, len(hex_digest), 2)]
    words = [hex_digest[i:i+8] for i in range(0, len(hex_digest), 8)]
    def reverse_bits_in_byte(byte_hex: str) -> str:
        b = int(byte_hex, 16)
        rb = int(f"{b:08b}"[::-1], 2)
        return f"{rb:02x}"
    return {
        "chars_rev": hex_digest[::-1],
        "bytes_rev": "".join(reversed(bytes_)),
        "words32_rev": "".join(reversed(words)),
        "bits_each_byte_rev": "".join(reverse_bits_in_byte(x) for x in bytes_),
    }

def topological_pin_scan(msg: bytes):
    tr = sha256_trace(msg)
    rows = []
    for r in tr.rounds:
        rows.append({
            "t": r["t"],
            "W_pop": popcount(r["W"]),
            "K_pop": popcount(r["K"]),
            "T1_pop": popcount(r["T1"]),
            "T2_pop": popcount(r["T2"]),
            "carry_T1_pop": popcount(r["carry_T1"]),
        })
    df = pd.DataFrame(rows)
    df["pin_score"] = df["carry_T1_pop"] + np.abs(df["T1_pop"] - df["T2_pop"])
    return tr, df
```

```

def glass_key_candidates(candidates):
    rows = []
    for name, msg in candidates:
        digest = hashlib.sha256(msg).hexdigest()
        vars_ = digest_reverse_variants(digest)
        rows.append({
            "name": name,
            "bytes": len(msg),
            "digest": digest,
            "digest_pop": popcount(int(digest, 16)),
            "chars_rev_pop": popcount(int(vars_["chars_rev"], 16)),
            "bytes_rev_pop": popcount(int(vars_["bytes_rev"], 16)),
            "words32_rev_pop": popcount(int(vars_["words32_rev"], 16)),
        })
    return pd.DataFrame(rows)

def basin_summary(words):
    rows = []
    for i in range(0, len(words)-1, 2):
        a, b = words[i], words[i+1]
        xor_term, carry_term, sum_term = kernel_add32(a, b)
        rows.append({
            "pair": f"{i}/{i+1}",
            "xor_pop": popcount(xor_term),
            "carry_pop": popcount(carry_term),
            "sum_pop": popcount(sum_term),
        })
    return pd.DataFrame(rows)

pin_trace, pin_df = topological_pin_scan(b"BASE|EMITTER|COLLECTOR|TRUE|MIRROR|")
display(pin_df.head(10))
display(glass_key_candidates(harmonic_inputs()).head(10))
display(basin_summary(STD_K[:16]))

```

	t	W_pop	K_pop	T1_pop	T2_pop	carry_T1_pop	pin_score
0	0	11	13	15	12	8	11
1	1	15	14	11	21	8	18
2	2	12	20	17	17	5	5
3	3	16	20	15	12	8	11
4	4	12	16	15	15	8	8
5	5	16	16	18	16	3	5
6	6	15	14	18	17	5	6
7	7	13	18	22	15	6	13
8	8	14	14	19	12	9	16
9	9	0	11	14	18	5	9

	name	bytes	digest	digest_pop	chars_rev_pop	bytes_rev_pop	words32_rev_pop
0	h00	52	41898e022833fc6bbe07ec9b8f0871c9519f547751d78a...	124	124	124	124
1	h01	74	61dfdd5e77b728cc9554e279a4a214446f49491a6b28f5...	134	134	134	134
2	h02	44	98453336a4646ac0c1e16a3b2ee46ff9d0f6ba13b05287...	128	128	128	128
3	h03	32	fad50ed11bf63a34bb3f930d1a3ccac95677a23ad4a477...	136	136	136	136
4	h04	38	cab130eca9e2b6f3a7cea07b4121ecad5df160fc86f2c4...	132	132	132	132
5	h05	52	49896ca831bb619b52ff96446cd81d96f830d68cd7e108...	125	125	125	125
6	h06	74	837d0f91a8f95b7170d935c522fdda6ee78ffc0d1041b5...	133	133	133	133
7	h07	44	2076ce01dc776580832c96a3d677a6b82309cb3091febe...	125	125	125	125
8	h08	32	a45b5ad1edfd94909a16127cea80aaf2f12fa97c5835f6...	127	127	127	127
9	h09	38	a5e28017442e239785d5798896268ac1ea82a552309b78...	125	125	125	125



	pair	xor_pop	carry_pop	sum_pop
0	0/1	17	5	15
1	2/3	14	12	21
2	4/5	16	8	15
3	6/7	16	7	19
4	8/9	15	5	15
5	10/11	20	5	14
6	12/13	15	11	19
7	14/15	20	6	20

XV. Closing section

This notebook does not try to prove every ontological statement in the paper. What it does provide is a **large executable substrate** aligned to the paper's actual sections:

- attractor geometry,
- BBP-style addressing,
- tri-transistor basin mechanics,
- anti-phase cancellation,
- dual-wave storage,
- SHA crystallization,
- anomaly measurement,
- reverse peeling,
- backpressure control,
- silicon/carbon parity.

It is meant to sit next to the paper as an operational companion.